

# Seguridade

## Introducción

A seguridade da información é algo fundamental para calquera organización. Normalmente céntrase en garantir:

- Confidencialidade: Existen datos privados que deben estar protexidos ante accesos non autorizados.
- Dispoñibilidade: Non deben producirse denegacións de acceso a datos sobre os que hai dereito de acceso.
- Integridade: Non deben producirse modificacións non adecuadas, nin danos da información.

Desde un punto de vista da aplicación de normas de seguridade, esta verase influenciada por:

- Lexislación vixente: Aspectos legais, como o dereito a dispoñer de determinada información, confidencialidade etc. As leis como a LOPD (Lei orgánica de protección de datos) é máis recentemente o RXPDP (Regulamento Xeral de Protección de Datos) son de obrigado cumprimento.
- Normas específicas da organización: Pode clarificarse a información en distintos niveis de seguridade. Isto determina que usuarios teñen acceso sobre cada dato, e que pode facer sobre el.

## ISO / IEC 27001

**ISO/IEC 27001** é un estándar da seguridade da información que permite establecer os requisitos necesarios para crear, manter e mellorar un Sistema de Xestión de Seguridade da Información (SXI) dunha organización.

Establece un conxunto de boas prácticas para permitir ás organizacións realizar unha correcta xestión dos riscos e a aplicación de procedementos e controis necesarios para contrarrestar, ou incluso eliminar, eses riscos.

A análise de riscos da organización estará baseada nun inventario de activos, un inventario de ameazas, e a definición dun nivel de risco adaptable para a organización.

Poñerá unha serie de plans de accións para contrarrestar eses riscos.

Está baseada no ciclo de Deming ou PDCA (Plan-Do-Check-Act):

- Plan: Establecer obxectivos e procesos, e facer unha planificación temporal.
- Do: Implementar eses obxectivos e procesos.
- Check: Verificar o éxito ou fracaso das medidas tomadas.
- Act: Recopilar o aprendido na fase anterior e poñelo en marcha. A veces aparecen recomendacións que iniciarían un novo ciclo.

## Seguridade nas bases de datos

A seguridade impleméntase nos SXBD para protexer a BD de ameazas e ataques, sexan intencionados ou accidentais.

Os ataques ou accesos indebidos poden producir perda de:

- Confidencialidade.
- Dispoñibilidade.
- Integridade (incluíndo roubo ou fraude).

Podemos implementar medidas para contrarrestar ou eliminar os riscos:

- Control de acceso: Autenticación e autorización.
- Control de Acceso Obrigatorio.
- Control de Acceso Discrecional.
- Outras medidas:
  - BD estáticas
  - Cifrado de datos.
  - Auditoría...

## Autenticación e autorización

### AuthIDs

- Un *Authorization identifier* pode ser:
  - Un identificador ou nome de usuario.
  - Un nome dun rol
  - **PUBLIC** é un *pseudo-AuthID* que referencia a totalidade dos AuthIDs (presentes e futuros) da BD.
- Identificador de usuario.
  - É un identificador de SQL que se utiliza para conectarse á base de datos.
  - Conceptualmente identifica un usuario, persoa ou programa que accede á base de datos.
  - O estándar non especifica como crear estes nomes de usuarios.
  - Exemplos de varios SXBD:

- Oracle:
  - Con identificador: `CREATE USER "nome.usuario" IDENTIFIED BY "contrasinal";`
  - Identificación externa: `CREATE USER usuario IDENTIFIED EXTERNALLY;`
- PostgreSQL:
  - `CREATE USER "nome.usuario" WITH PASSWORD 'contrasinal';`
- SQL Server:
  - Usuario da BD: `CREATE USER usuario WITH PASSWORD='contrasinal';`
  - Usuario baseado en login: `CREATE USER usuario FROM LOGIN usuariowindows;`

## Autenticación

A autenticación é o primeiro proceso que realiza o SXBD para realizar o control de acceso dos usuarios á base de datos.

Consiste en identificar a un usuario e verificar a súa identidade.

O proceso implica:

1. Identificarse: o usuario debe indicar o seu *AuthId*.
2. Debe verificarse, con información adicional, que o usuario é quen di ser. Isto faise normalmente utilizando unha contrasinal. A verificación pode ser:
  - Realizada completamente polo SXBD (que almacena usuarios coas súas contrasinais - cifradas-).
  - Delegando a verificación no Sistema Operativo.
  - Delegando a verificación noutro sistema externo.

Unha vez o usuario está autenticado, pasaríamos á segunda fase: a autorización, que indica o que o usuario pode facer.

## Autorización

A autorización especifica o que un usuario (en xeral, un *authID*) pode facer na base de datos. Debemos ter en conta que o feito de que un usuario se autentique correctamente, non ten por que darlle dereito a nada, nin siquiera a conectarse á BD.

Existen fundamentalmente 2 tipos de mecanismos de control de acceso:

- Control de Acceso Obrigatorio (*MAC: Mandatory Access Control*).
  - O estándar SQL non inclúe soporte para este control de acceso.
  - O nome pode ser enganoso, xa que non é o máis habitualmente implementado nos SXBD.

- Control de Acceso Discrecional (*DAC: Discretionary Access Control*)
  - Está definido no estándar SQL.
  - Está baseado na concesión e revocación de privilexios, utilizando as sentencias **GRANT** e **REVOKE**.
  - Normalmente inclúe a autorización baseada en roles.

## Control de Acceso Obligatorio (MAC)

Está baseado en políticas a nivel de sistema (non modificables por usuarios individuais).

- A cada obxecto da base de datos asígnaselle unha *clase de seguridade*.
- A cada usuario asígnaselle un *nivel de autorización* para cada clase de seguridade.
- Un usuario pode ler ou escribir un obxecto baseándose nos niveis de autorización e clases de seguridade.

Trata de garantir que os datos confidenciais non pasen a un usuario sen o nivel de seguridade axeitado.

## Control de Acceso Discrecional (DAC)

### Privilexios

O control de acceso discrecional está baseado na concesión e revocación de privilexios. Existen 2 tipos de privilexios: a nivel de sistema e a nivel de obxecto.

#### Privilexios de sistema ou de servidor:

- Especifican accións que un usuario pode levar a cabo, sen estar vinculadas con ningún obxecto en concreto.
- Non está definido no SQL estándar, pero practicamente todos os SXBD os inclúen.
- Normalmente, a xestión é sintácticamente igual á xestión de roles.

#### Privilexios a nivel de obxecto:

Están asociados a un obxecto en concreto, e dependerán do tipo de obxecto.

Están definidos no SQL estándar (aínda que os xestores utilizan habitualmente privilexios adicionais)

Para táboas e vistas:

```
SELECT [<lista de columnas>]
INSERT [<lista de columnas>]
DELETE
UPDATE [<lista de columnas>]
```

```
REFERENCES [<lista de columnas>]
TRIGGER [<lista de columnas>]
```

Existen outros, como `USAGE` (para dominios) ou `EXECUTE` (para procedimientos almacenados).

O creador dun obxecto ten todos os privilexios sobre ese obxecto e a potestade de pasar eses privilexios a outros.

A política de seguridade de SQL indica que non debemos poder inferir información á que non temos acceso. Se un usuario pretende seleccionar datos dunha táboa á que non ten acceso, o SXBD debería indicar que tal táboa non existe.

## Control de privilexios sobre obxectos da BD

Utilízase a sentencia `GRANT` :

```
GRANT { <lista_privilexios> | ALL PRIVILEGES }
      ON <obxecto>
      TO <lista_authids>
      [WITH GRANT OPTION]
```

Un usuario só pode conceder un privilexio sobre o se ten ese privilexio e a otestade de pasalo a outros.

- O creador dun obxecto ten todos os privilexios sobre el, e a potestade de pasalos a outros.
- A sentecia `GRANT` permite conceder varios privilexios á vez, pero só un único .
- ``ALL PRIVILEGES` é a lista de todos os privilexios aplicables ó tipo de obxecto implicado.
- `<lista_aithids>` é unha lista (separada por comas) de AuthIDs (que poden ser usuarios, roles ou o pseudo-authID `PUBLIC` ).
- Utilizando `WITH GRANT OPTION` concédese tamén a potestade de pasar ese privilexio a outros.

Exemplos:

```
GRANT SELECT, DELETE, INSERT ON tab1 TO usuario1;
GRANT UPDATE(sal) ON emp TO scott, role345 WITH GRANT OPTION;
GRANT ALL PRIVILEGES ON EMP TO theboss;
GRANT SELECT ON emp TO PUBLIC;
```

## Revocación de privilexios sobre obxectos da BD

Utilízase a sentencia `REVOKE` :

```
REVOKE [GRANT OPTION FOR] { <lista_privilexios> | ALL PRIVILEGES }
ON <obxecto>
FROM <lista_authids>
{ CASCADE | RESTRICT }
```

- Un usuario só pode revocar un privilexio que concedeu explicitamente cunha sentenza **GRANT** previa.
- Usando **GRANT OPTION FOR** só se retira a potestade para pasar os privilexios a outros (pero os privilexios consérvanse).
- Se algún usuario da <lista\_authids> propagou algún privilexio a outros:
  - **CASCADE** retira o privilexio en cascada a todos os AuthIds implicados.
  - **RESTRICT** produce un erro e non revoca os privilexios.

Exemplos:

```
REVOKE DELETE, INSERT ON tab1 FROM usuario1;
REVOKE UPDATE(sal) ON emp FROM scott CASCADE;
REVOKE SELECT ON emp FROM PUBLIC;
```

## Liás e cadeas de concesións de privilexios

Ao executar a sentenza **GRANT** concedendo un privilexio a un AuthID establécense unha liña de concesión.

Cando se propaga un privilexio a terceiros, poden crearse complexas cadeas de concesión.

Un AuthId perde un privilexio se se lle retira por todas as liñas de concesión.

```
-- u1 é o propietario dunha táboa xenérica "t"

1. (u1) GRANT SELECT ON t to u2, u3 WITH GRANT OPTION;
-- u2 dá permisos a u2 e u3, coa opción grant.

2. (u2) GRANT SELECT ON t to u4;
3. (u3) GRANT SELECT ON t to u4;
-- Tanto u2 como u3 poden dar o permiso a u4.
-- Cada sentenza GRANT dá lugar a unha liña
-- de concesión.

4. (u3) REVOKE SELECT ON t FROM u4;
-- u3 revoca a súa liña de concesión, pero
-- u4 mantén o privilexio debido á liña de
-- concesión desde u2.
```

```
5. (u2) REVOKE SELECT ON t FROM u4;
-- u2 revoca a súa liña de concesión.
-- Agora u4 perde o privilexio.
```

Outro exemplo:

```
-- Os puntos 1-3 son iguais á transparencia anterior

1. (u1) GRANT SELECT ON t to u2, u3 WITH GRANT OPTION;
2. (u2) GRANT SELECT ON t to u4;
3. (u3) GRANT SELECT ON t to u4;

4. (u1) REVOKE SELECT ON t FROM u3 CASCADE;
-- u1 revoca o privilexio a u3 coa opción en cascada
-- u3 perde o privilexio, e a súa liña de concesión
-- a u4 desaparece.

5. (u1) REVOKE GRANT OPTION FOR SELECT ON t FROM u2 CASCADE;
-- u1 revoca a "grant option" a u2 en cascada.
-- u2 mantén o privilexio pero perde a potestade de
-- pasalo. A súa liña de concesión a u4 desaparece.
-- Agora u4 perde o privilexio.
```

## Roles

Un rol é un tipo especial de AuthID que se utiliza para facilitar a xestión dos privilexios.

Características dun rol:

- É un AuthID que non pode conectarse á BD.
- Podemos concederlle privilexios (ou outros roles).
- Podemos asignar ("conceder", grant) o rol a outros AuthIDs (usuarios, roles, PUBLIC), co que pasan a ter os privilexios do rol concedido.

Exemplo:

```
CREATE ROLE facturador;
CREATE ROLE xestor;

GRANT ALL PRIVILEGES ON factura TO facturador;
GRANT SELECT ON artigo TO facturador;

GRANT ALL PRIVILEGES ON artigo TO xestor;
GRANT facturador TO xestor;
```

```
GRANT facturador TO facturador1, facturador2;
GRANT xestor TO xestor1;
REVOKE facturador FROM facturador2;
REVOKE REFERENCES ON factura FROM facturador;

DROP ROLE xestor;
```

A concesión de roles utiliza tamén sentenzas GRANT e REVOKE.

```
GRANT <rol> [, <rol>, ...]
    TO <lista_authids>
    [WITH ADMIN OPTION]

REVOKE [ADMIN OPTION FOR] <rol> [, <rol>, ...]
    FROM <lista_authids>
```

Para os roles, estas sentenzas **GRANT** e **REVOKE** non inclúen ningunha cláusula **ON <obxecto>**, e ademais utilizan opcionalmente a cláusula **WITH ADMIN OPTION**.

O funcionamento desta cláusula non é exactamente igual a **with grant option**.

Neste caso, ademais do rol, pásase a potestade de *administrar* este rol:

- Permite conceder privilexios ou outros roles a ese rol.
- Permite conceder o rol, ou revocallo, a calquera lista de AuthIDs.
- Permite eliminar (drop) ese rol.

## Técnicas adicionais de seguridade

Ademais do control de acceso obrigatorio e discrecional, extendido mediante o uso de roles, podemos aplicar outras técnicas que melloran a seguridade nas nosas bases de datos

Entre elas:

- Vistas (+ control de acceso discrecional).
- Bases de datos estáticas.
- Cifrado da información
- Prevención de inxección SQL.
- Copias de seguridade
- Auditoría

## Vistas como elemento de seguridade



É habitual querer limitar o acceso a certas filas ou certas columnas dunha táboa.

Non sempre é posible dar os permisos directamente sobre a táboa.

Solución:

- Retirar os privilexios sobre a táboa (se estaban concedidos).
- Crear unha vista coa consulta desexada. Son comúns:
  - Vista vertical: Selección de todas as filas pero só parte das columnas (atributos).
  - Vista horizontal: Selección de todos os atributos pero só parte das filas.
- Dar permisos sobre a vista. Se é necesario poden usarse roles.

Exemplo:

```
CREATE VIEW emp20
  AS SELECT empno, ename, job, mgr, hiredate, sal, comm, deptno
  FROM emp
  WHERE deptno=20
  WITH CHECK OPTION;

GRANT ALL PRIVILEGES ON emp20 TO usr;
```

## Bases de datos estáticas

Contén datos individuais, pero só publica datos agregados.

É unha forma de "anonimizar" e non publicar información confidencial.

Exemplos:

- Salarios medios por profesión e zona.
- Datos sanitarios.

Posible problema: a inferencia de datos confidenciais a partir dos agregados.

Habitualmente, non se publicarán datos se non proveñen dun certo número mínimo de tuplas.

## Cifrado de información

As comunicacións co SXBD deben estar cifradas, especialmente se o cliente se conecta a través de internet. Úsase habitualmente SSL/TLS e certificados.

Poden cifrarse tamén os datos da BD, xa que hai información que ninguén (nin o DBA) debe coñecer.

Exemplos:

- Contraseñas dos usuarios da BD, ou de usuarios de aplicacións que se almacenen na BD.
- Información confidencial, como un historial clínico.

Pode protexerse a información ante accesos externos (ex: roubo dun ficheiro da BD, posibilidade de obter información do arquivo "crudo").

Mecanismos:

- Para as contrasinais é común cifrar externamente a información e almacenar só a clave cifrada. O habitual é un cifrado unidireccional (HASH, SHA-1, MD5, ...). No proceso de login, cífrase a clave introducida e compárase coa almacenada.
- Para o caso de información confidencial como o historial clínico debe usarse un cifrado bidireccional, para poder recuperar a información cifrada.

## Prevención de inxección SQL

A inxección de SQL é un problema de seguridade debido fundamentalmente a non controlar de xeito correcto a entrada de datos nos programas que acceden á BD.

A continuación móstrase un exemplo cun programa (mal) codificado en Python.

```
query = "select datanac from persoa where name = '{}'"
nomepers = input('Introduce nome: ')
# Introducimos: Ada Byron
# Parece correcto:
# query.format(nomepers) => "select datanac from persoa where nome = 'Ada Byron'"
cursor.execute(query.format(nomepers)) # => Ok
# Pero, OLL0!
# introducimos nome: '; DROP TABLE persoa; --
query.format(nomepers) # "select datanac form persoa where name = '; DROP
TABLE persoa; --'"
cursor.execute(query.format(nomepers)) # => Elimina a táboa persoa
```

Unha posible solución consiste en utilizar parámetros

```
query = "select datanac from persoa where nome = %(nomepers)s"
nomepers=input('Introduce nome: ')
# introducimos nome: '; DROP TABLE persoa; --
# Buscaría unha persoa chamada: '; DROP TABLE persoa; --
# Non ocorre a inxección SQL
cur.execute(query, {'nomepers' : nomepers})
```

## Backups

Os backups ou copias de seguridade son tamén unha ferramenta fundamental desde o punto de vista da seguridade.

Pode producirse perda de información por varios motivos:

- Borrado (accidental ou intencionado) por parte dun usuario.
- Mal funcionamento do software (SO, SXBD, outros programas, poderían producir a corrupción dun ficheiro ou sistema de ficheiros).
- Fallo hardware dos dispositiovs.

É importante, polo tanto, dispoñer dun plan global de salvaguarda do sistema, e realizar probas periódicas para comprobar o seu correcto funcionamento.

## **Auditoría**

### **Introducción**

Como activos importantes da nosa organización, para protexer os nosos datos utilizamos:

- Seguridade (a priori). Por exemplo, creando usuarios e roles para asignarlles privilexios.
- Auditoría de base de datos: Permite rexistrar información.
  - Dos eventos globais do sistema (arranque, parada, condicións de erro, ...).
  - Das accións que os usuarios fan sobre os obxectos da BD.
- Analise dos logs da auditoría (a posteriori): permite descartar
  - Accesos, ou intentos de acceso non autorizados.
  - Abuso no acceso a datos.

### **Que podemos auditar?**

**Auditoria básica:**

1. Acceso de usuarios
2. Uso de privilexios de sistema
3. Modificacións do esquema da base de datos
4. Modificacións de datos sensibles.

**É posible auditar:**

- Todos os privilexios de sistema.
- A nivel de calquera obxecto da base de datos (táboa, vista)
  - Diversas accións: select, delete, insert, update
  - Para intentos con éxito, sin éxito ou ambos.
  - A nivel de usuario individual ou global (todos os usuarios).

### **Auditoría en oracle**

## 1. Oracle audit

- Sentenzas DDL, xestión de usuarios, ...
- Sentenzas DML: A nivel de táboa.

## 2. Triggers de sistema

- Eventos do sistema: arranque-parada da base de datos.
- Conexión/desconexión de usuarios.
- Modificación do esquema.

## 3. Triggers de sistema

- poden chegar a nivel de fila e columna
- Só detectan modificacións (non hai trigger para SELECT)
- Aparte da auditoría podemos facer outras cousas (BD Activas)

## 4. Auditoría detallada

- Captura de accesos de lectura, ata nivel de fila e columna.
- Baseado en triggers internos.
- Usa o paquete `DBMS_FGA` (procedementos PL/SQL)
- Define "políticas" (policies) de auditoría.

## 5. Logs do sistema

- Alert logs: rexistran arranque-parada, cambios estruturais (engadir arquivo á BD), ...

# Oracle Audir

Permite auditar

- Todos os permisos que se poden dar ( `GRANT` ) a un usuario ou rol.
  1. Auditoría de sentencias (create table, create session, ...)
  2. Auditoría de privilexios (alter user, grant, ...)
  3. Auditoría de obxectos (select en táboas, ...)
- Exemplos:

```
audit alter table;  
audit create session;  
audit alter user;  
  
noaudit drop table;
```

Podemos comprobar no catálogo de Oracle o que estamos auditando:

```
select audit_option, success, failure  
  from dba_stmt_audit_opts  
 union
```

```
select audit_option, success, failure
       from dba_priv_audit_opts;
```

## Auditoria de sesiones

```
SELECT username, terminal, action_name, timestamp, logoff_time, returncode
FROM dba_audit_session;
```

## Triggers de sistema

Exemplo: creación de triggers para registrar acceso á BD.

```
CREATE TABLE CONEXIONS
(
    USUARIO CHAR(20),
    DATA_HORA TIMESTAMP,
    TIPO CHAR(12)
);

CREATE TRIGGER LOG_CONEXION
AFTER LOGON ON DATABASE
BEGIN
    INSERT INTO CONEXIONS
        VALUES (USER, SYSDATE, 'CONEXIÓN');
END LOG_CONEXION;
/

CREATE TRIGGER LOG_DESCONEXION
BEFORE LOGOFF ON DATABASE
BEGIN
    INSERT INTO CONEXIONS
        VALUES (USER, SYSDATE, 'DESCONEXIÓN');
END LOG_CONEXION;
/
```

Exemplo: análise do "log" creado polos tirrggers.

```
SQL> select * from conexions;
```

ningunha fila seleccionada

```
SQL> disconnect
[Desconnectadp]
```

```
SQL> connect usuario/*****
Conectado.
SQL> select * from conexions;
```

| USUARIO | DATA_HORA                | TIPO        |
|---------|--------------------------|-------------|
| usuario | 31/01/22 16:47:41,000000 | DESCONEXIÓN |
| usuario | 31/01/22 16:47:44,000000 | CONEXIÓN    |

## Triggers de inserción, borrado e modificación

Exemplo: queremos auditar os cambios ( `update` ) dos salarios dos empregados.

```
CREATE OR REPLACE TRIGGERS T_LOG_CAMBIO_SAL
AFTER UPDATE OF SAL ON EMP
FOR EACH ROW
BEGIN
    INSERT INTO LOGCAMBIOSAL
        (USUARIO, TIMESTAMP, EMPNO, SAL_ANT, SAL_NOVO)
    VALUES
        (USER, SYSDATE, :NEW.EMPNO, :OLD_SAL, NEW_SAL);
END;
```

## DBMS\_FGA: Fine Grain Auditing

Exemplo: quedarán rexistrados os accesos á táboa DEPT do usuario SCOTT, sempre que se acceda á columna DNAME e o seu valor de DEPTNO sexa 30.

```
begin
    dbms_fga.add_policy (
        object_schema=>'SCOTT',
        object_name=>'DEPT',
        policy_name=>'nome_ventas',
        audit_colum=>'DNAME',
        audit_condition=>'deptno = 30' );
end;

select timestamp, db_user, object_name, sql_text
from dba_fga_audit_trail;
```